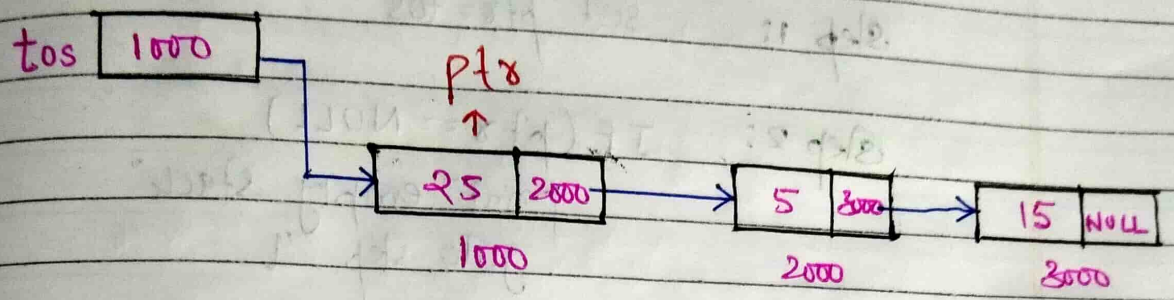
→ Linked list implementation ~~यसी~~ of stack :-



Algorithm :- To create stack :-

Step 1: Create nptr [new_node]

Step 2: set nptr → data = val

Step 3: set nptr → next = NULL

Step 4: IF val = -1
goto step 7

Step 5: IF (tos = = NULL)
5.1.) tos = nptr

Step 6: else,
6.1.) ptr = tos
6.2.) Repeat until ptr→next ! = NULL
ptr = ptr→next
6.3.) ptr→next = nptr

Step 7: Exit

Akash

→ Algorithm to display a stack;

Step 1: set ptr = tos

Step 2: IF (ptr == NULL)
print "empty stack"
goto step 4

Step 3: Repeat until (ptr != NULL)
print ptr→data and
ptr = ptr→next

Step 4; Exit()

→ // C program to create & display stack

```
#include<stdio.h>
#include<stdlib.h>
struct node
{
    int data;
    struct node *next;
};

struct node *create_stack(struct node *tos)
{
    struct node *ptr, *nptr;
    int val;
    printf("enter -1 to stop\n");
    scanf("%d", &val);
    while(val != -1)
    {
      nptr=(struct node*)malloc(sizeof(struct node));
        nptr->data=val;
        nptr->next=NULL;
      if( tos==NULL)
      {
            tos = nptr;
      }
      else
      {
            ptr=tos;
```

Akash

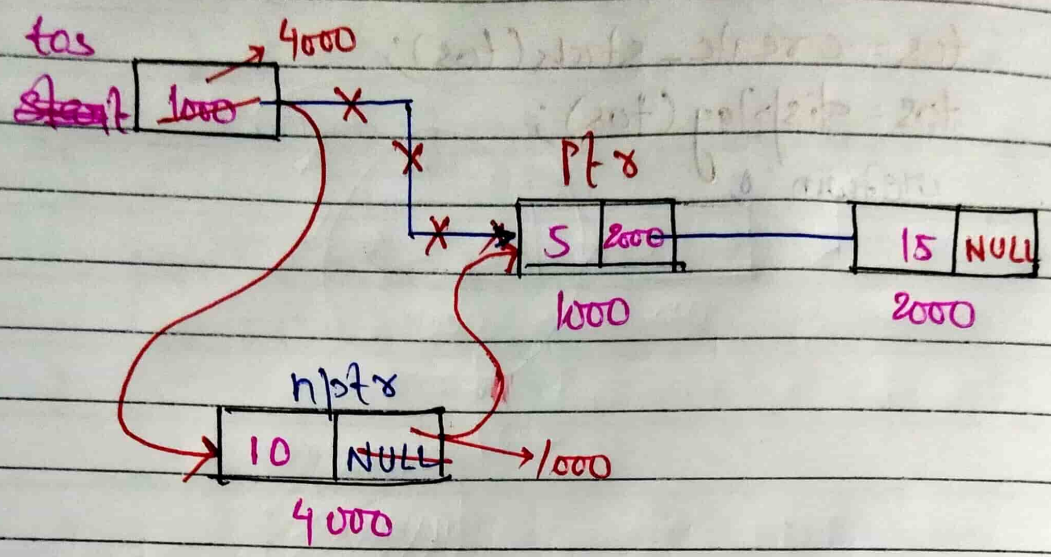
```
while(ptr->next != NULL)
  {
        ptr=ptr->next;
    }
      ptr->next=nptr;
  }
    printf("enter data\n");
    scanf("%d",&val);
  }
    return tos;
}

struct node *display(struct node *tos)
  {
        struct node *ptr;
        if (tos==NULL)
          {
              printf("stack is empty");
          }
        else
          {
              ptr=tos;
              while(ptr!=NULL)
                {
                  printf("%d->",ptr->data);
                    ptr=ptr->next;
                }
          }
        return tos;
  }
```

Akash

```
int main()
{
    struct node *tos = NULL;
    tos = create_stack(tos);
    tos = display(tos);
    return 0;
}
```

Akash

→ Push → It is same as insert at begining in Linked List.



Algorithm :-

Step 1: Create nptr[new-node]

Step 2: set ptr = ~~start~~ tos

Step 3: set ~~start~~ tos = nptr

Step 4: set nptr → next = ptr

Step 5: Exit

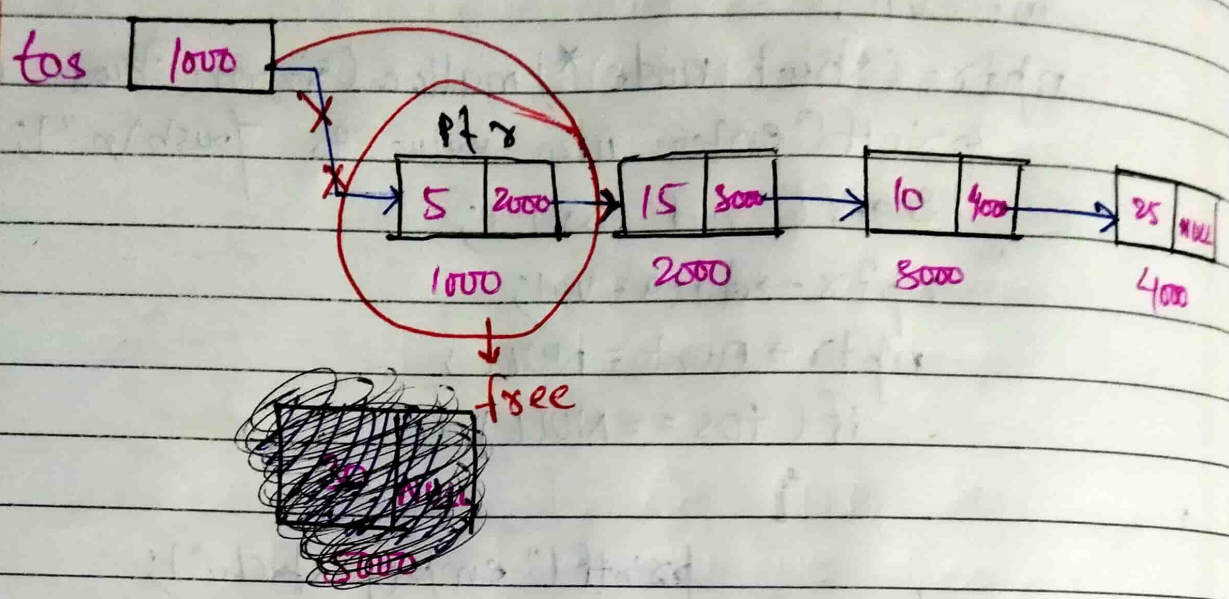
Akash

→ Pseudo code;

```
struct node *ptr, *nptr;
int val;
nptr = (struct node *) malloc (sizeof(struct node));
    printf("enter new value to push\n");
    scanf("%d", &val);
    nptr->data = val;
    nptr->next = NULL;
    if( tos == NULL)
    {
        printf("empty stack");
    }
else
    {
        ptr = tos;
        tos = nptr;
        nptr->next = ptr;
    }
return tos;
```

Akash

→ Pop:- It is same as deleting the first node in Linked List.



Algorithm:-

Step 1: IF (tos == NULL)
print "empty stack"

Step 2: else,
2.1.) ptr = tos
2.2) tos = ptr→next
2.3) free(ptr)

Step 3: Exit

→ pseudo code:

```
struct node *ptr;
if(tos== NULL)
  {
      printf("empty stack");
  }
else
  {
      ptr=tos;
      tos=ptr->next;
      free(ptr);
  }
  return tos;
}
```

Akash

→ Stack as ADT →

ADT → An abstract data type (ADT) is a mathematical model for a data type, which defines the behavior (operation) of the data structure and the rules for how these operation should functions, without specifying the implementation details.

In simple terms, An ADT tells you what a data structure does, not how it does it.

A Stack is an abstraction data type that follows the last in first out (LIFO) principle.

The main operation for the stack are:

- push(): add element to the top element of the stack.
- pop(): Remove and return the top element of the stack.
- peek(): Returns the top element without removing it.
- isEmpty(): check if the stack is empty.
- isFull(): (In case of fixed-size stack) Check if the stack is full.

Akash

→ Implementation →

1. Array
2. Linked - List

} we have does this before.

Akash